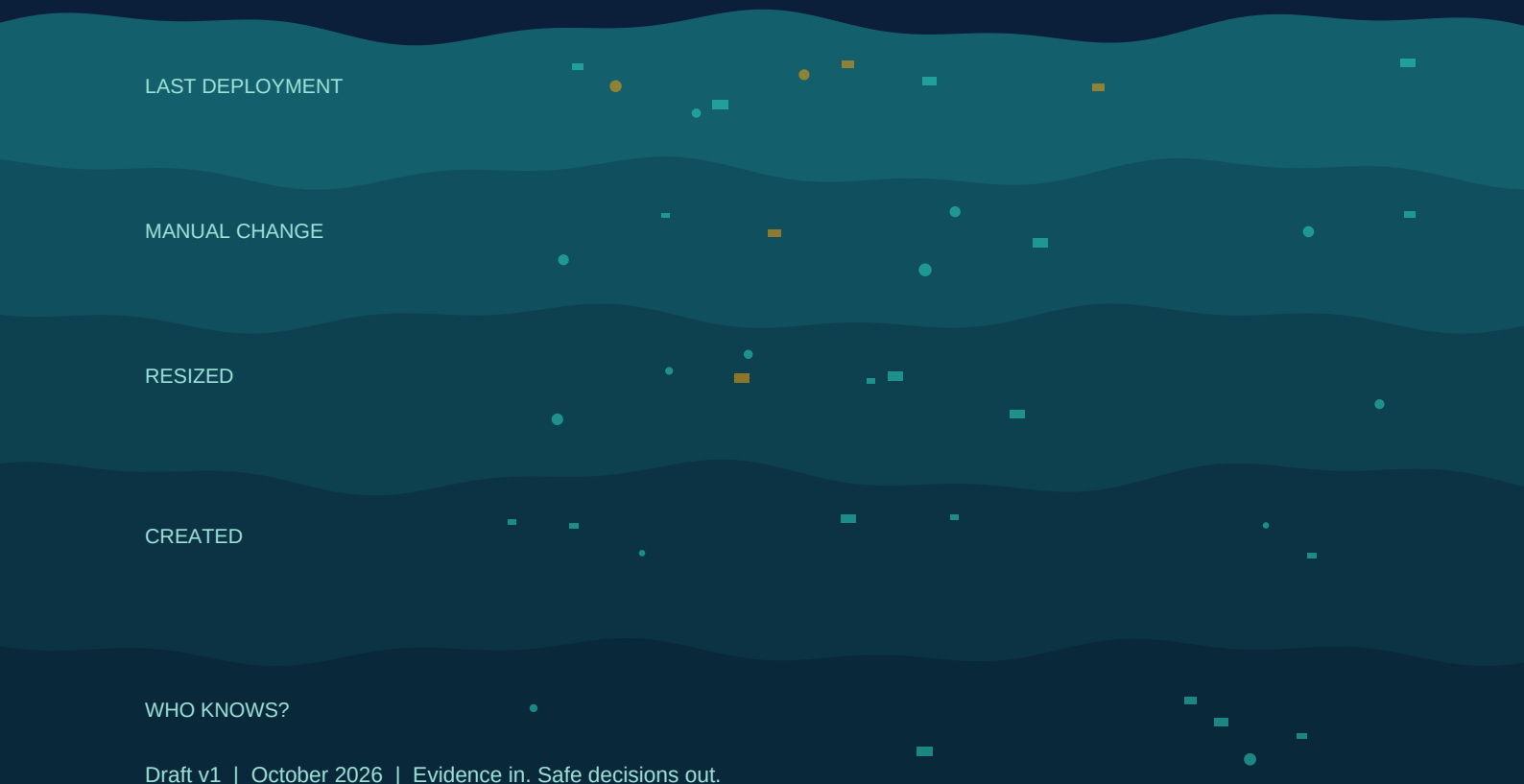
PROBLEM STATEMENT & MVP PLAN

Cloud Resource Archaeologist

Know why every cloud resource exists.
Retire it safely.

Why does it exist? **Who owns it?** **Can we delete it?**



Draft v1 | October 2026 | Evidence in. Safe decisions out.

1 The problem

A company runs thousands of cloud resources, and nobody can quickly answer the basic questions about them.

Why does this instance exist?
No tag, no ticket, no note.

Who owns this bucket?
The creator left two reorgs ago.

Can we delete this database?
Nobody dares to find out.

Answering today means searching the trail below, one system at a time. When nobody knows, the resource stays and keeps costing money, or someone deletes it and breaks something.

TODAY'S INVESTIGATION TRAIL

AWS > Terraform > GitHub > CI/CD > CloudTrail > Jira > Docs > Slack

Someone who has been here 5 years

The gap
Inventory, cost, drift and cleanup tools each show part of the picture. What stays unsolved is the **decision**. A resource that looks idle may still support a production workload, a backup process or an undocumented dependency, so people do not trust a "safe to delete" label.

2 The solution

An **evidence-based history** for every resource, used to run a **reversible decommission process**, so deleting becomes a low-risk operation.

A Provenance

- Joins AWS, Terraform, GitHub, CI/CD, CloudTrail, cost and usage
- Builds a timeline for each resource
- Gives an owner with a confidence level

B Risk verdict

- Answers "can I delete this?" with reasons
- Checks dependencies, traffic, last use, owner and saving
- Never deletes automatically

C Reversible retirement

- Snapshot and quarantine first
- Watch period, then rollback or delete
- Every step in an audit trail

The provenance timeline

Every resource gets a story. The example below is illustrative.

RESOURCE CARD (ILLUSTRATIVE)	
Resource	EC2: payment-api-prod
Owner	Payments Platform Team
Source	modules/ec2/main.tf
Cost	$187 / month
Usage	CPU 8%, low network, 0.4 req/s
Needs	Security group, IAM role, RDS, load balancer

The risk verdict: "Can I delete this?"

The tool investigates instead of answering yes or no.

DELETE RISK: MEDIUM — **Potential saving: $187 / month**

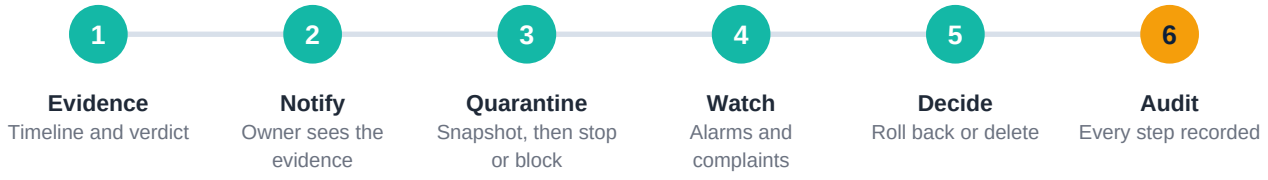
Evidence for deleting
No meaningful traffic for 21 days. CPU stays near 8%.

Evidence against
Referenced by a production Terraform module and attached to the payment load balancer.

Required action: owner approval, then staged retirement. Never automatic.

Reversible decommissioning: the differentiator

Instead of keep or delete, resources move through a staged process that can be undone until the very end.



3 What makes it different

The AI explains and orchestrates, but every decision rests on **evidence from cloud APIs, Git and audit logs**, shown with confidence levels. The product is the evidence and the safe workflow, not a chatbot.

Existing category	What it does well	What stays open
Inventory and cost tools	Show what exists and what it costs	Do not explain why it exists or whether it is safe to remove
Cleanup scanners and policy engines	Find idle or orphaned resources; some apply approved removals	Deleting is still a leap of faith without a reversible step
Creator and ownership trackers	Attribute a resource to a creator, commit or pipeline	Stop at attribution: no risk verdict, no staged retirement
Cloud Resource Archaeologist	Joins provenance, a risk verdict and reversible retirement with an audit trail	Early idea: needs a working prototype and a check against existing tools

Related tools that cover parts of this include CloudSlash, tf-why, Stakpak, CleanCloud, Cloud Custodian and Firefly. This comparison is based on public information; the combined loop is the claim to validate, not a proven first.

4 MVP scope

IN SCOPE

- One AWS account; EC2, EBS volumes and security groups
- Terraform and GitHub for attribution
- A CloudTrail trail with long retention (event history alone covers only 90 days)
- CloudWatch metrics for usage
- Provenance timeline, owner guess with confidence, risk verdict
- Reversible flow for EBS and EC2, with owner approval
- Evidence coverage on every answer; verdicts blocked when evidence is thin
- Creator and owner kept separate, with a fallback order for finding the owner
- Confirm-or-correct button that stores ground truth
- Simple UI or command line with a question box

OUT OF SCOPE FOR NOW

- Other clouds
- Slack, Jira and wiki ingestion
- S3 and RDS
- Deleting without approval
- Cost allocation at scale

Demo script (about 5 minutes)

1	Ask "Why does this instance exist?" and show the provenance timeline.
2	Show a thin-evidence resource: the tool says "insufficient evidence" and lists what is missing.
3	Ask "Can I delete this?" and show the risk verdict with its reasons.
4	Start decommissioning an idle test resource: snapshot, stop, watch.
5	Trigger a rollback, then show the audit trail.
6	Show the savings summary.

Success measures

Accuracy	Owners confirm the "why it exists / who owns it" answer on 20 real resources. Report the percentage.
Coverage	Share of resources explained, and share honestly flagged as unknown.
Safety	Zero unintended outages; every quarantine reversible.
Value	Monthly savings from retired resources, and hours saved per investigation.

Risks and safeguards

RISK	SAFEGUARD
Resources older than the CloudTrail window	Use a long-retention trail or AWS Config history; mark older ones "unknown origin"
Hidden dependencies and seasonal jobs	Never auto-delete; quarantine with a watch period; owner approval
Wrong ownership guess	Show confidence and evidence; allow correction
Imprecise per-resource cost	Use cost data with resource IDs where available; label estimates
Excess permissions	Read-only analysis; a separate, narrowly scoped role for quarantine actions

Build plan (about 4 weeks)

WEEK 1
Connect AWS (read-only), CloudTrail and GitHub; inventory the three resource types; measure owner coverage

WEEK 2
Provenance engine and timeline for EC2

WEEK 3
Risk verdict, then the reversible quarantine flow for EBS and EC2

WEEK 4
Accuracy test with owners, demo and numbers

Verify before building

1. Sandbox or non-production AWS account with CloudTrail retention you control.
2. Overlap with existing IBM tools such as Concert, Turbonomic and Cloudability.
3. IP terms and approval for access to company repos and cloud accounts.
4. Baseline: in one real account, count resources with no owner tag. That number is the strongest evidence for the pitch.

5 Where it can fail, and how we handle it

Five weak spots, one theme: **never sound more confident than the evidence.**

Weak spot	What goes wrong	What we do
1. Messy or missing clues	Logs expire, Git history is squashed, Terraform state drifts, some resources are made by hand	Show coverage ("2 of 6 clues found"). Mark each clue found, negative, unavailable or conflicting. Below a minimum, answer "insufficient evidence", never "low risk"
2. Creator is not the owner	The person who deployed in 2022 may have left or changed teams	Keep creator and owner as separate fields. Use a fixed fallback order (below). Escalate to a review queue
3. Vague "watch for problems"	No definition of a problem, a watch period or how a call to the resource is detected	A concrete detection method per resource type, a watch window based on risk, and a strict rule: no detection signal, no quarantine
4. Too many clouds	The data sources (CloudTrail, CloudWatch, EC2 APIs) are AWS-specific	AWS only for version one. Keep the data model cloud-neutral; other clouds are a separate later phase
5. Confidence is just a number	"75% sure" means nothing unless it has been tested	Use High / Medium / Low from clear rules, check them against owner confirmations, and report how often High was right

Finding the owner: fallback order

Owner tag > **CODEOWNERS** > **Account owner** > **Creator, if still active** > **Review queue**

Each answer says which step it came from. Cost-centre tag, CMDB lookup and creator's manager come later.

Quarantine: how problems are detected

Resource	Reversible quarantine	How a problem is detected
EC2 instance	Snapshot, then swap to a deny-all security group or stop it	Rejected connections in flow logs; failed health checks in dependent systems
EBS volume	Snapshot, then wait	Attach attempts in CloudTrail
S3 and RDS (later)	Deny-all policy or security group, plus snapshot. Stopping RDS alone is not stable (AWS restarts it after about 7 days, as far as we know)	Denied requests in access logs; rejected connections

What counts as a problem: a rejected inbound attempt from a source not on the allowlist, a denied call that references the resource, an alarm in a dependent system, or an owner asking for it back. **Watch window:** about 7 days for unattached volumes with no dependents, 14 to 30 days for medium risk, and a full month-end or quarter-end for high risk. Annual jobs cannot be fully guarded against, and the tool says so.

> **No detection signal, no quarantine**
> If the signal needed to detect a problem (for example flow logs) is not enabled, the tool declines the quarantine or requires stronger approval.

Checking the confidence

- Labels come from rules. High means at least two independent strong clues agree and none conflict. The AI model is not asked for a confidence number.
- Hand-verify 20 to 50 resources as ground truth, and save every owner confirmation or correction as a labeled example.
- Report the result, for example: "Of 20 High answers, owners confirmed 18." If High is often wrong, fix the rules before anyone relies on it.
- Consider numeric scores only after about 100 labeled cases per category.

> **Added to the MVP**
> Evidence coverage on every answer with verdicts blocked on thin data; creator and owner kept separate with a fallback order; the quarantine detection table and rule; a confirm-or-correct button that stores ground truth.